

NOUS PERSONAL AGENT OS

Vue 架构 → React 架构

转换说明

范围：frontend/ (Vite + TypeScript)

原则：复用原功能、原路由、原 CSS 渲染效果

后端 FastAPI / LangGraph / Skill 体系零改动

文档日期：2026-08-15

目录

1. 核查结论与工作范围
2. 转换原则：功能与渲染如何原样复用
3. 技术栈对照
4. 目录结构对照
5. 框架核心概念映射
6. 路由对照
7. 页面与组件逐文件对照
8. 状态、生命周期与事件映射
9. Markdown / 高亮 / 导出链接的渲染复用
10. 功能对等清单
11. 视觉与动效复用清单
12. 明确未改动的部分
13. 验证方法
14. 结论

1. 核查结论与工作范围

对仓库 **Nous** 做了完整扫描：根目录含 backend、frontend、docs、scripts；frontend 源码全部为 **.tsx / .ts / .css**，**不存在任何 .vue 单文件组件**，package.json 依赖为 React 18.3.1、react-dom、react-router-dom 6.28、@vitejs/plugin-react、react-markdown、remark-gfm、rehype-highlight、highlight.js。README 技术栈一节已写明「React 18.3 · TypeScript 5.7 · Vite 6 · React Router 6」。

结论：当前工作区里的前端已经是 React 架构，不是 Vue 工程。因此本次交付不是再把一套 Vue 源码改写成 React，而是以现有 React 实现为权威目标，完整记录「等价 Vue 3 架构 → 当前 React 架构」的映射，并核对功能与 CSS 渲染是否 1:1 保留。后端、数据库、Agent、Skill 包格式均不在转换范围内。

扫描依据

检查项	结果
*.vue 文件	0 个
vue / vue-router / pinia / @vitejs/plugin-vue	未出现在 frontend/package.json
入口	frontend/src/main.tsx → ReactDOM.createRoot + BrowserRouter
构建插件	vite.config.ts 使用 @vitejs/plugin-react
页面	Chat / Conversations / Skills / SkillDetail / Settings，均为 function component
样式	frontend/src/styles/index.css 单一全局样式表，BEM 命名
后端	FastAPI + LangGraph，与前端框架无关，保持原样

表 1 仓库核查结果

2. 转换原则：功能与渲染如何原样复用

从 Vue 迁到 React 时，最容易丢掉的不是「能不能跑」，而是交互细节和视觉节奏：侧栏呼吸光晕、消息入场、输入框自动增高、Markdown 代码块复制按钮、导出图片内联预览等。本项目的转换策略是把「逻辑」和「皮」拆开：逻辑用 **React** 语法重写，皮（class 名 + CSS + 字体 + 动效）原文件复用。

原则	做法	效果
CSS 零改写	继续使用 styles/index.css 与原 BEM class (sidebar_brand、msg--user、hero_chip 等)	颜色、圆角、阴影、动画关键帧与 Vue 模板渲染一致
路由 path 不变	/、/chat/:id、/conversations、/skills、/skills/:id、/settings	书签、后端无链接、Skill 详情跳转全部可继续使用
API 契约不变	仍走 /api/v1，X-User-Id: local-dev，错误信封 {error.code/message/request_id}	后端零改动；Vite proxy 仍把 /api 转到 8000
组件职责不变	Layout / MessageList / SkillEditor 等边界与 Vue SFC 一一对应	功能点可按文件对照验收，而不是打散重写
无 UI 框架	不引入 Ant Design / MUI / Tailwind，避免皮肤漂移	手写 CSS 的视觉语言完整保留

原则	做法	效果
类型下沉	Vue 的 props/emits 变成 TypeScript interface; API 类型集中在 api/types.ts	tsc --noEmit 可在转换后做回归

表 2 转换原则

3. 技术栈对照

层	等价 Vue 3 方案	当前 React 方案
运行时	Vue 3.4 + Composition API	React 18.3.1
语言	TypeScript + <script setup lang="ts">	TypeScript 5.7 + TSX
构建	Vite 6 + @vitejs/plugin-vue	Vite 6.0.5 + @vitejs/plugin-react 4.3.4
路由	vue-router 4 (createWebHistory)	react-router-dom 6.28 (BrowserRouter)
状态	组件内 ref/reactive; 跨页无需 Pinia	组件内 useState/useRef; 无 Redux/Zustand
HTTP	封装 fetch / axios composable	src/api/client.ts 统一 fetch 封装
Markdown	markdown-it + markdown-it-gfm	react-markdown + remark-gfm
代码高亮	highlight.js + vue 指令/插件	rehype-highlight + highlight.js github.css
样式	单文件 <style> 或全局 CSS	全局 styles/index.css (class 名保持 BEM)
入口挂载	createApp(App).use(router).mount('#app')	createRoot(#root).render(<BrowserRouter> <App />)

表 3 技术栈对照。两边都刻意不用大型状态库——页面状态都是局部的。

4. 目录结构对照

Vue 工程习惯 views/ + components/ + composables/ + api/。当前 React 工程几乎是同一张地图，只把 views 改名为 pages，把 .vue 改成 .tsx。这是刻意的：目录级复用能让验收按文件打勾。

Vue 3 等价路径	React 实际路径	职责
src/main.ts	src/main.tsx	挂载、引入全局 CSS 与 highlight 主题
src/App.vue	src/App.tsx	Layout + router-view / Routes
src/views/ChatView.vue	src/pages/ChatPage.tsx	对话、乐观更新、关闭沉淀
src/views/ConversationsView.vue	src/pages/ConversationsPage.tsx	会话列表、筛选、抽取、删除
src/views/SkillsView.vue	src/pages/SkillsPage.tsx	Skill CRUD、包导入、检索探针
src/views/SkillDetailView.vue	src/pages/SkillDetailPage.tsx	Skill 详情与编辑
src/views/SettingsView.vue	src/pages/SettingsPage.tsx	模型供应商运行时配置
src/components/AppLayout.vue	src/components/Layout.tsx	侧栏导航 + 健康状态
src/components/MessageList.vue	src/components/MessageList.tsx	欢迎英雄区、气泡、打字指示
src/components/MessageInput.vue	src/components/MessageInput.tsx	Enter 发送、自适应高度
src/components/MarkdownContent.vue	src/components/MarkdownContent.tsx	GFM + 代码块 + 导出媒体

Vue 3 等价路径	React 实际路径	职责
src/components/ExecutionTrace.vue	src/components/ExecutionTrace.tsx	Agent 步骤折叠时间线
src/components/SkillBadge.vue	src/components/SkillBadge.tsx	命中 Skill 与 0/0 反馈
src/components/SkillEditor.vue	src/components/SkillEditor.tsx	新建/编辑 Skill 模态框
src/components/ProviderEditor.vue	src/components/ProviderEditor.tsx	供应商表单与连通性测试
src/api/*	src/api/*	chat / conversations / skills / llmConfig / types
src/lib/exportLinks.ts	src/lib/exportLinks.ts	导出文件 URL 归一化，框架无关
src/styles/index.css	src/styles/index.css	全部视觉与关键帧，原样复用
index.html #app	index.html #root	挂载点改名，字体 link 不变

表 4 文件级映射。lib 与 api、CSS 属于框架无关层，转换时直接搬迁。

5. 框架核心概念映射

这是语法层对照。转换时按左列在 **Vue** 源码里定位，再在右列用 **React** 等价物替换，不要引入新的抽象（例如不要为了「更 React」突然上 Redux）。

Vue 3	React 18	本项目中的实例
<script setup> SFC	export default function Xxx()	所有页面与组件
ref(false) / reactive({})	useState(...)	pending、messages、editorOpen
computed(() => ...)	渲染期内派生 / useMemo	closed = status === 'closed'
watch(id, load)	useEffect(() => { ... }, [id])	ChatPage 按 routeId 拉会话
onMounted(fetchHealth)	useEffect(..., [])	Layout 拉 /health
onUnmounted 取消请求	useEffect 返回 cancelled 标志	ChatPage / SkillDetailPage
defineProps / defineEmits	interface Props + 回调	SkillEditor.onSave / onClose
v-model	value + onChange	搜索框、textarea、select
v-if / v-else	条件 && 或三元表达式	closed 会话禁用输入
v-for="item in list" :key	list.map(item => ... key=)	消息、表格、能力包卡片
@click="fn"	onClick={fn}	所有按钮
@keyup.enter	onKeyDown 判断 Enter	MessageInput: Enter 发送、Shift+Enter 换行
<slot />	children: React.ReactNode	Layout 包住 Routes
<router-link> / <router-view>	<NavLink> / <Routes> <Route>	App.tsx + Layout
useRouter() / useRoute()	useNavigate() / useParams()	跳到 /chat/:id
\$refs.textarea	useRef<HTMLTextAreaElement>	输入框高度重置
nextTick 滚到底	useEffect + scrollIntoView	MessageList 跟新消息
v-html 渲染 Markdown	react-markdown 组件树	禁止危险 HTML，走 AST

表 5 概念映射。Markdown 从 v-html 改为组件树，安全性更好，class 仍走原 CSS。

入口对照

Vue: createApp(App).use(router).mount('#app')。React: ReactDOM.createRoot(#root).render(StrictMode + BrowserRouter + App)。StrictMode 在开发态会故意双调用 effect，因此所有 useEffect 都用 cancelled 标志或忽略过期响应，避免竞态。

样式挂载对照

两边都在入口 import './styles/index.css'，并额外引入 highlight.js/styles/github.css。index.html 里 Google Fonts (Manrope + Syne) 的 <link> 原样保留。因此品牌字体、侧栏深色渐变、卡片玻璃拟态 (backdrop-filter) 都不因框架切换而丢失。

6. 路由对照

vue-router 的 routes 表与 React Router 的 Route 组件一一对应。通配符 Vue 用 path:('/:pathMatch(.*)*)'，React 用 path="*" 配 Navigate 回首页。

Path	Vue 组件	React 组件	行为
/	ChatView	ChatPage	新会话；发送成功后 replace 到 /chat/:id
/chat/:id	ChatView	ChatPage	加载历史消息、Skill 溯源、关闭沉淀
/conversations	ConversationsView	ConversationsPage	分页、搜索、状态筛选
/skills	SkillsView	SkillsPage	列表 + 市场 + zip/JSON 导入
/skills/:id	SkillDetailView	SkillDetailPage	详情、启用/禁用、编辑
/settings	SettingsView	SettingsPage	当前生效模型 + 供应商 CRUD
*	redirect /	Navigate to /	未知路径回工作台

表 6 路由表。NavLink 的 end={to==='/'} 对应 Vue 的 exact，避免工作台在子路径也高亮。

7. 页面与组件逐文件对照

7.1 Layout.tsx (原 AppLayout.vue)

侧栏品牌、四项导航、能力闭环文案、底部健康点。Vue 用 router-link-active class, React 用 NavLink 的 className 回调拼 nav-link--active。onMounted 调 getHealth() 变成 useEffect。data-ok 属性驱动 CSS 绿点，选择器未改。

7.2 ChatPage.tsx

核心对话页。保留：路由无 id 时空白新会话；有 id 时拉详情；乐观插入用户气泡，失败回滚；成功后以服务端消息为准刷新；used_skills 写入 skillsByMessage；关闭会话触发抽取并提示；已关闭会话只读并引导新开。Vue 的 watch(() => route.params.id) 对应 useEffect([routeId])。

7.3 ConversationsPage.tsx

分页 PAGE_SIZE=15、标题搜索、状态筛选、抽取 Skill、删除确认。徽章 class (badge--active / --closed / --failed / --draft) 完全复用 CSS。

7.4 SkillsPage.tsx

功能面最宽：列表筛选、新建/编辑/启停/删除、重建索引、检索探针、内置包一键导入、JSON 粘贴导入、zip (nous-pack/2) 预览授权安装、已安装包启停卸载。模态框仍是 SkillEditor; open 为 false 时直接 return null，对应 Vue 的 v-if="open"。

7.5 SkillDetailPage.tsx

统计卡片、关键词胶囊、workflow 有序列表 (CSS counter)、Agent 指令、来源案例、元信息。草稿态 alert--warning 文案与样式未改。

7.6 SettingsPage.tsx

当前生效配置、供应商表、.env 兜底、Bedrock SDK 检查、三路测试结果槽位 (编辑器 / 当前配置 / 行内) 避免互相覆盖。密钥字段在编辑器里「空=保持原值」。

7.7 对话子组件

组件	必须保住的交互
MessageList	空态英雄区四条建议 chip; 用户/助手气泡; 助手 Markdown; 执行轨迹; Skill 徽章; token 数; 复制按钮 hover 才显现; pending 三点跳动 + 三步占位; 自动滚到底
MessageInput	自适应高度上限 200px; Enter 发送、Shift+Enter 换行; disabled 时不可发
MarkdownContent	GFM 表格/任务列表; 围栏代码语言标签+复制; 行内 code; 导出路径改写成「点击下载」; 图片导出内联预览
ExecutionTrace	默认对最新助手消息展开; 步数与工具次数摘要; ok/error/info 点状状态
SkillBadge	跳转详情; 相似度分数; 0/0 只投一次; 失败不打断主界面

表 7 对话链路组件的验收要点

8. 状态、生命周期与事件映射

以 ChatPage 为例的状态表

Vue	React	含义
const conversationId = ref(null)	useState<string null>	当前会话
const messages = ref([])	useState<Message[]>([])	消息列表
const pending = ref(false)	useState(false)	发送中
const skillsByMessage = reactive({})	useState<Record<...>>	按消息挂 Skill
watch(routeId, load)	useEffect(..., [routeId])	切会话时重载
router.replace(...)	navigate(..., { replace: true })	首次发送后写 URL

表 8 ChatPage 状态对照

请求竞态

Vue 3 常用 AbortController 或 stale 标志; 本项目 React 实现统一用 let cancelled = false, 在 effect cleanup 里置 true, then 回调先判断再 setState。这对应 Vue onBeforeUnmount 取消, 避免 StrictMode 双挂载把旧会话写进新页面。

表单 v-model 拆解

Vue 的 `v-model="search"` 在 React 拆成 `value={search}` 与 `onChange` 里 `setSearch + setOffset(0)`。SkillEditor 在 `open/skill` 变化时用 `useEffect` 重置字段，对应 Vue `watch([open, skill], fillForm, { immediate: true })`。

9. Markdown / 高亮 / 导出链接的渲染复用

助手气泡是视觉差异最大的区域。Vue 常见路径是 `markdown-it` 出 HTML，再用 `v-html` 塞进 `.md`，`highlight.js` 扫 `pre code`。React 改为 AST：`react-markdown` → `remark-gfm` → `rehype-highlight` → 自定义 components。DOM class 仍指向同一套 CSS：`.md`、`.md-code`、`.md-code_header`、`.md-inline-code`、`.md-table-wrap`、`.msg_image`。

视觉/行为点	CSS / 逻辑如何保住
深色代码块 + 语言标签 + 复制	CodeBlock 组件输出 <code>md-code</code> 结构，复制成功加 <code>md-code__copy--done</code>
表格横向滚动	table 包一层 <code>md-table-wrap</code>
引用条	<code>.md blockquote</code> 左边框 <code>accent</code> ，未改 CSS
导出文件链接	<code>normalizeExportHref</code> 把 <code>localhost</code> 绝对 URL 收成同源 <code>/api/v1/files/exports/...</code>
生成图预览	<code>isExportImage</code> 时同时给「查看原图」链接和 <code>img.msg_image</code>
用户纯文本换行	非助手消息走 <code>renderPlainContent</code> ，保留 <code>pre-wrap</code> ，只 <code>linkify</code> URL
语法高亮配色	入口引入 <code>highlight.js/styles/github.css</code> ，再被 <code>.md-code .hljs</code> 覆盖为深底浅字

表 9 渲染层复用。exportLinks.ts 是纯函数，Vue/React 可共用。

10. 功能对等清单

下列每一项在当前 React 源码中均有对应实现，转换验收可按此表勾选。

模块	功能	状态
工作台	空态英雄文案与四条建议 chip 一键发送	已复用
工作台	发消息、乐观用户气泡、失败回滚	已复用
工作台	助手 Markdown、执行轨迹、Skill 徽章与反馈	已复用
工作台	打字指示与三步 pending 占位	已复用
工作台	关闭并沉淀 Skill；已关闭只读	已复用
工作台	导出文件下载链接与图片内联	已复用
会话	分页、搜索、状态筛选	已复用
会话	手动抽取 Skill / 删除确认	已复用
Skills	列表、状态筛选、启用禁用、编辑删除	已复用
Skills	检索探针显示向量分+关键词分	已复用
Skills	内置包导入、JSON 导入	已复用
Skills	zip 预览、权限勾选、安装/启停/卸载	已复用

模块	功能	状态
Skills	重建向量索引并回显统计	已复用
Skill 详情	统计、关键词、步骤、指令、案例、元信息	已复用
模型	当前生效卡片、测当前、回落 .env	已复用
模型	供应商 CRUD、行内测试、编辑器测试	已复用
模型	OpenAI 兼容 + Bedrock 表单、检查 boto3	已复用
布局	导航高亮、健康点、模型名、响应式侧栏	已复用
基础设施	Vite /api 代理、错误信封、network_error 文案	已复用

表 10 功能对等清单 (19 项全部保留)

11. 视觉与动效复用清单

渲染效果不靠 React 重写，而靠「同一份 CSS 继续作用在同一套 class 上」。JSX 只负责把 className 拼对。下面是必须肉眼核对的动效与皮肤。

选择器 / 关键帧	效果
body + @keyframes atmosphere	背景三层径向渐变缓慢位移
.sidebar::before + glow-drift	侧栏底部青绿光斑左右漂移
.nav-link:hover / --active	hover 右移 2px; 激活项青绿渐变内描边
.brand-mark	36px 圆角方标 + 内芯光晕
.hero-in	欢迎区 0.55s 上浮淡入
.hero_chip:hover	上移 2px 并改边框色
.msg-in	每条消息 0.25s 入场
.msg_typing-dots / typing-dot	三点错相跳动
.msg:hover .msg_actions	复制按钮默认透明，悬停显现
.pack-card:hover	上移 3px
.btn:hover	上移 1px + 轻阴影
.spinner / spin	加载圆环
.card backdrop-filter	面板毛玻璃
@media (max-width: 840px)	侧栏改顶栏，胶囊与页脚隐藏
CSS 变量 --accent #0f8f86 等	整站色板与圆角、阴影 token

表 11 视觉验收表。转换后不应重写这些规则，只核对 class 是否还在 JSX 上。

12. 明确未改动的部分

- 后端 FastAPI、LangGraph 图、Skill 提取链、混合检索、工具注册表。
- API 路径、错误信封、X-User-Id、分页参数。
- docker-compose、frontend/nginx.conf、Dockerfile（仍托管 Vite 构建产物）。

- docs/examples 与 nous-pack/2 能力包格式。
- 全局 CSS 与 Google Fonts 引入。
- 纯函数 lib/exportLinks.ts 与 api/types.ts 的字段形状。

转换边界只在 UI 运行时：模板语法、响应式、路由库、Markdown 库。任何「趁机重构后端 / 换 UI 框架 / 改 class 名」都会破坏渲染对等，故禁止。

13. 验证方法

静态

- cd frontend && npm run typecheck (tsc --noEmit)
- cd frontend && npm run build (类型检查 + Vite 生产构建)
- 确认产物不含 vue 运行时，index.html 入口为 /src/main.tsx

手工回归（与后端联调）

- 打开 http://localhost:5173，侧栏健康点为绿或「后端未连接」文案正确。
- 点建议 chip 发一条消息：乐观气泡 → pending 三点 → 助手 Markdown。
- 关闭会话，到会话页看抽取状态，到 Skills 看草稿/新 Skill。
- Skills 页检索探针、导入包、zip 预览勾选权限。
- 模型页新增供应商、测试连接、切回 .env。
- 缩到 840px 以下，侧栏应变顶栏，对话仍可滚动发送。

对比手法

若手头有旧 Vue 构建：同一后端、同一条会话，对欢迎区、气泡、代码块、侧栏、设置页做截图叠加。允许的差异只有滚动条、字体 hinting、React StrictMode 开发态双请求；不允许色值、圆角、文案、动效丢失。

14. 结论

Nous 前端的目标架构是 **React 18 + TypeScript + Vite + React Router**，用函数组件和 Hooks 替换 Vue SFC / Composition API，用 JSX 条件与 map 替换指令，用 react-markdown 替换 v-html。为保住「原来能有的功能和渲染效果」，转换遵守三条硬约束：

- 路由与信息架构不变；
- API 层与类型不变；
- CSS 选择器与关键帧不变，JSX 只负责把 class 接回去。

当前仓库核查结果：上述目标架构已经落地，frontend/src 下 5 个页面、8 个组件、6 个 API 模块均是 React/TS 实现，无 Vue 残留。本文档作为 Vue → React 的对照说明书，可用于答辩、交接或后续若从 Vue 分支再迁一次时的施工清单。

项	数量 / 说明
React 页面	5 (Chat / Conversations / Skills / SkillDetail / Settings)
React 组件	8 (Layout、消息链 5 个、两个 Editor)

项	数量 / 说明
API 模块	6 (client、chat、conversations、skills、llmConfig、types)
全局样式	1 份 index.css, 约 1600 行, 原样复用
后端改动	0
功能对等项	19 / 19 保留

表 12 交付快照

附录 A 依赖替换一览

移除 (Vue 侧)	加入 (React 侧)	备注
vue	react + react-dom	运行时
vue-router	react-router-dom	history 模式等价
@vitejs/plugin-vue	@vitejs/plugin-react	Vite 插件
vue-tsc	tsc -b / tsc --noEmit	tsconfig jsx: react-jsx
markdown-it 生态	react-markdown + remark-gfm + rehype-highlight	GFM 能力对齐
highlight.js (可保留)	highlight.js + github.css	主题 CSS 继续用

表 13 依赖替换。highlight.js 本身框架无关，两端都能用。

附录 B 典型代码形状对照（示意）

Vue 模板：<div v-if="closed" class="badge badge--closed">已关闭</div>。React：{closed && 已关闭}。class 字符串完全相同，所以徽章颜色不会漂。

Vue：<router-link :to="/skills/\${id}">。React：<Link to={`/\${skills}/\${id}`}> 或 navigate(`/skills/\${id}`)。Skill 详情、会话标题点击行为保持不变。

Vue 列表：<article v-for="m in visible" :key="m.id" :class=["msg", `msg--\${m.role}`]>。React：visible.map(message => <article key={message.id} className={`msg msg--\${message.role}`}>)。msg--user / msg--assistant 的背景与头像渐变因此得以保留。